

Are these leads already in a sequence?

A build brief for the Lead Scanner session: flag, at upload time, which leads are already being worked in Apollo. Every API fact below was confirmed by a real call against Jack's account on 30 September 2026 — none of it is inferred. Two of these facts silently produce wrong code if guessed; they are marked.

What to build

When a CSV of leads is uploaded to the Scanner, show for each row whether that person is **already in one of Jack's or Carly's Apollo sequences**, what happened to them, and where they stand — so the same lead is not re-worked from scratch, and so a lead that was scanned but never sequenced is visible as such.

Per lead, the target row reads:

- **Never sequenced** — scanned and uploaded, no cadence ever started
- **In sequence** — which one, active or finished, and which step it reached
- **Standing** — the Apollo contact stage and the last call outcome
- **Notes** — the Scanner's own scan verdict, which already travels into Apollo

Why this is possible at all

The Scanner's verdict is already inside Apollo. Contacts uploaded from the Scanner carry it in Apollo custom fields, so matching a lead back to its scan needs no access to the Scanner's IndexedDB — which would be impossible anyway, since each Artifact has its own origin and cannot read another's storage.

```
typed_custom_fields: {
  "68abe22a26a866001d3e1a08": "Dynamics 365",
  "68abe2198cb839001d7d67c3": "Pitch D365 Sales Pro (Act Now, High fit) · runs 0365, Azure · High priority (70)"
}
```

Custom field ids → names

Field id	Name	Type	Carries
68abe22a26a866001d3e1a08	Lead Solution Area	string	Scanner product line: "Dynamics 365" / "M365 / Azure"
68abe2198cb839001d7d67c3	Notes	textarea	Scan pitch, fit and priority score
68c32705c53494000d84f430	Lead Score	number	Numeric score
68c988cf882b300021d96942	Product Line	textarea	Freer-text product line

The call that answers the question

`apollo_contacts_search` returns the email *and* the sequence membership in one record, which is what makes a single pass sufficient.

```
"email": "justin@legacylabor.com",
"emailer_campaign_ids": ["6a0b653aba6c9100208889c0", "6a8c57f7bb0f38001460ab11"],
"last_activity_date": "2026-09-30T17:24:34.000+00:00",
"contact_stage_id": "6a73bd924f135d0014ac11a0",
"label_ids": ["6a749ad3f6b704000191faff"],
"contact_campaign_statuses": [{
  "emailer_campaign_id": "6a0b653aba6c9100208889c0",
  "status": "finished",
  "inactive_reason": "Completed last step",
```

```

"added_at": "2026-09-08T16:40:05.978+00:00",
"finished_at": "2026-09-21T15:09:48.122+00:00",
"current_step_position": 6,
"send_email_from_email_address": "jack.snellgrove@wired-cio.com"
}]

```

An empty `emailer_campaign_ids` is the “never sequenced” signal. `contact_campaign_statuses` gives status, step reached and which mailbox sent — everything needed for “where it stands”.

Traps — things that look like they work and do not

1. A task record carries no email

`apollo_tasks_search` is the obvious way to list who is in a sequence. Do not use it for matching. A task’s contact is `{id, name, linkedin_url}` — **no email and no company**. The Scanner keys contacts on email first with a name+company fallback, so a task roster cannot be matched back to a Scanner lead without a second lookup per contact. This is why the design goes contact-first.

2. Analytics answers in markdown, not rows

`apollo_analytics_sync_report` returns a rendered markdown table as a string under summary. There is no structured row array. Parse the table, and if parsing fails show “unavailable” rather than zero — a zero here reads as “no activity” and is a lie.

3. Filters that do not exist

Wanted	Reality
Filter contacts by sequence	No campaign filter on <code>contacts_search</code> . Fetch and filter client-side.
Filter contacts by created date	No date filter. Sort by <code>contact_created_at</code> desc and stop at the cutoff.
Filter contacts by custom field	Not supported. <code>q_keywords</code> covers name, title, employer, email only.
Export all contacts to CSV	<code>csv_exports_export_view</code> only exports custom-object collections, not contacts.
Read Apollo workflow objects	No workflow/automation endpoint exists in the connector at all.

4. The one filter that does exist, and is the whole trick

`contact_label_ids` is a server-side filter. Jack’s Apollo Plays already maintain lists that bucket leads by exactly the standing we want, so fetching by list beats crawling 19,667 contacts: roughly 90 targeted calls instead of 197 blind pages.

Plays-maintained lists (contacts)

List	List id	Count	Owner
Unprocessed Leads	68abe5c95d714d00190c607d	5,823	Russell — verify scope
No Answer — Auto Email Continue	6a749ad3f6b704000191fa9f	802	Jack
Meeting Booked Contacts	6a02523197f4590001552bf0	102	Jack
SalesHive Non Actioned Leads	69efcdacf6d770001d045465	880	Jack
May Leads	69f8cd829693db0021be518d	584	Jack
May 7th	69fe04615c4a720011fd42dd	506	Jack
Nurturing Campaign MSP Camp	6924952b5103a60021420bcb	184	Jack

List	List id	Count	Owner
Saleshive Dynamics Backlog	69fa1314448bfe001df5024e	55	Jack

The first three are written by Apollo's rules engine (`RulesEngine::ActionHandler#upsert_labels`) and update continuously. **“Unprocessed Leads” is owned by Russell's user id, not Jack's** — confirm in Apollo whether it is team-wide before treating it as Jack's never-contacted pile.

Reference ids

People

Who	Apollo user id	Email
Jack Snellgrove	68bf4ba5f68a0600194acd11	jack@wiredcio.com
Carly Parks	6a738903389433001cdab5da	carly@wiredcio.com
Team	66203bd7183e310422980459	—

Resolve these by **email** at run time rather than hardcoding the ids — an id can change and a silently empty result looks like “no work” rather than a broken filter. If neither resolves, say the filter failed.

Live sequences — Jack and Carly only

Sequence	Sequence id	Steps	Owner
Jack Main Sequence	6a0b653aba6c9100208889c0	6	Jack
Dual Effort Leads	6a15fa4ba24719001c6a860f	5	Jack
Jack Outbound Emails	6a6a9356b1e5bf000c6855f6	3	Jack
CSP Leads	6aad4a8c59b8f300184954da	5	Jack
Carly Main Sequence	6a8c57f7bb0f38001460ab11	6	Carly
Carly Outbound Emails	6aa1691e1da2db001c122762	3	Carly

Eight further sequences belong to other people on the team and are deliberately out of scope, per Jack: “just jack and carly that is it.”

Call outcomes Apollo actually uses

Apollo names this dimension “Disposition”. It matches the Scanner's own nine almost exactly, plus one the app does not carry. Counts are the last 3 months.

Reached them	Calls	Didn't reach them	Calls
Meeting Booked	77	No Answer	6,261
Call Back Scheduled	111	Gatekeeper / Front Desk	182
Info Requested	69	Wrong Number	72
Not interested	163	Left Voicemail	24
Do Not Contact	1	—	—
Qualified, Pending Review *	14	—	—

* exists in Apollo, absent from the Scanner's disposition set. Apollo labels carry stray trailing spaces — trim before matching. Two further outcome ids return as bare 24-hex strings (125 calls): those outcomes were deleted while calls still reference them. Render as “deleted”, never as a raw id.

The numbers this is meant to fix

Last 6 months, whole account:

Metric	Value	
Leads loaded	19,667	
Added to a sequence	8,194	11,473 never sequenced (58%)
Actually touched	7,972	11,695 never touched (59%)
Called	7,047	
Emailed	5,785	
Replied	114	1.4% of touched

Nearly six in ten qualified leads have never been called once. That is the gap the upload-time flag is meant to stop widening.

Suggested implementation

The Scanner already holds every lead ever uploaded — its Contacts store captures every row of every CSV, not only Strong Signal ones — and already has a disposition filter, a tier filter and a workedFilter (all / worked / unworked) driven by `isWorked()` in `lib/contacts.ts`.

The problem to fix: `isWorked()` reads `callCount` || `emailCount`, and those only ever count activity logged *inside the app*. Apollo says 7,047 contacts were called; the app says almost none. So the existing filter currently reports nearly every lead as never contacted, which is wrong.

New fields on Contact

Keep these separate from the app's own manual counters, for the same reason `disposition` is kept separate from `outreachStatus`: one is what we logged here, the other is what Apollo did. Then widen `isWorked()` to consider both.

```
apolloContactId?: string;
apolloSequences?: { id: string; name: string; status: string;
  step: number; addedAt: string }[];
apolloStanding?: string; // from the Plays list membership
apolloLastActivityAt?: string | null;
apolloSyncedAt?: string;
```

The upload-time check

Hook it into `App.tsx`'s `recordHistory` — the single choke point every CSV upload already passes through (Scanner direct upload, Library upload-into-folder, and Library “Load into Scanner” all funnel there, which is where `mergeContacts` already runs).

- Match on email first, then normalised name+company — the same convention `buildContactIndex` / `lookupContact` already use.
- Make the Apollo call **explicit**, never automatic, matching the shipped “Enrich via Apollo” pattern and Jack's standing rule: “I don't want too much going on in the background I can't see.”
- Make it resumable and stoppable, with a visible progress count.
- All of these calls are reads and consume **zero Apollo credits**.

Artifact manifest

capabilities is a full-set declaration — anything not restated on a republish is silently revoked. This has already broken every CSV download once. Restate **both** every time:

```
capabilities: {
  downloads: true,
  mcp: { servers: [{ server: "Apollo.io", tools: [
    "apollo_people_match", "apollo_people_bulk_match",
    "apollo_organizations_enrich", "apollo_organizations_bulk_enrich",
    "apollo_contacts_search", "apollo_labels_index",
    "apollo_emailer_campaigns_search", "apollo_users_search"
  ] }] }
}
```

The last four are the additions this feature needs. Everything else is already declared and must be carried forward.

Known gaps, stated rather than hidden

- **Exact per-contact call counts are not bulk-available.** Sequence membership and status are. Getting “called 3 times” needs `apollo_phone_calls_search` with a contact id — one call per contact, so it belongs in the contact detail view, not a bulk sync.
- **Left Voicemail is logged 24 times against 6,261 No Answers.** Voicemails are being left and not dispositioned, so No Answer is a bucket rather than an outcome and the true connect rate is understated.
- **No sequence finishes on a connected call.** `mark_finished_if_phone_call_connected` is unset on every sequence with call steps, so leads who book a meeting by phone stay enrolled and keep being worked. Fix in Apollo, not in code.
- **Nothing blocks re-entry by stage.** `excluded_contact_stage_ids` is empty everywhere, which is why the Not Interested stage holds 116 contacts but shows 145 sequence adds.
- **Apollo exposes no sequence delete.** Only create and update, and update uses declarative diff: any step id omitted from the payload is **deleted**. Treat writes as out of scope — per Jack, this is oversight, not an Apollo replacement.

Companion tool: **Apollo Sequence Control** — a standalone read-only console over the same data (sequences, workflow structure, dispositions, contact stages, mailbox health), scoped to Jack and Carly. Source: `apollo-control/index.html`. That tool and this brief were built from the same verified calls.